Section 19 / 30

Network Services

When working with Linux, managing various network services is essential. Proficiency in handling these services is crucial for several reasons. Network services are designed to perform specific tasks, many of which enable remote operations. It is important to have the knowledge and skills to communicate with other computers over the network, establish connections, transfer files, analyze network traffic, and configure these services effectively. This expertise allows us to identify potential vulnerabilities during penetration testing. Additionally, understanding the configuration options of each service enhances our overall comprehension of network security.

Consider a scenario where we are conducting a penetration test and encounter a Linux host being assessed for vulnerabilities. By monitoring network traffic, we observe that a user on this Linux host is connecting to another server via an unencrypted FTP server. Consequently, we are able to capture the user's credentials in plain text. This situation would be much less likely if the user were aware that FTP does not encrypt connections or the data transmitted. For a Linux administrator, this represents a critical oversight, as it not only exposes security weaknesses within the network but also reflects poorly on the administrators responsible for maintaining the network's security.

While it is not feasible to cover every network service, we will focus on the most important ones. This approach is beneficial not only for administrators and users, but also for penetration testers who need to understand the interactions between different hosts and their own systems.

SSH

Secure Shell (`SSH`) is a network protocol that allows the secure transmission of data and commands over a network. It is widely used to securely manage remote systems and securely access remote systems to execute commands or transfer files. In order to connect to our or a remote Linux host via SSH, a corresponding SSH server must be available and running.

The most commonly used SSH server is the OpenSSH server. OpenSSH is a free and open-source implementation of the Secure Shell (SSH) protocol that allows the secure transmission of data and commands over a network.

Administrators use OpenSSH to securely manage remote systems by establishing an encrypted connection to a remote host. With OpenSSH, administrators can execute commands on remote systems, securely transfer files, and establish a secure remote connection without the transmission of data and commands being intercepted by third parties.

Install OpenSSH

```shellsession
athane@htb[/htb]$ sudo apt install openssh-server -y
```

To check if the server is running, we can use the following command:

Server Status

```shellsession
athane@htb[/htb]$ systemctl status ssh

● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/system/system/ssh.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2023-02-12 21:15:27 GMT; 1min 22s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
   Main PID: 7740 (sshd)
      Tasks: 1 (limit: 9458)
     Memory: 2.5M
        CPU: 236ms
     CGroup: /system.slice/ssh.service
             └─7740 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
```

As penetration testers, we use OpenSSH to securely access remote systems when performing a network audit. To do this, we can use the following command:

SSH - Logging In

```shellsession
athane@htb[/htb]$ ssh cry0l1t3@10.129.17.122

The authenticity of host '10.129.17.122 (10.129.17.122)' can't be established.
ECDSA key fingerprint is SHA256:bKzhv+n2pYqr2r...Egf8LfqaHNxk.

Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

```
Warning: Permanently added '10.129.17.122' (ECDSA) to the list of known hosts.

cry0l1t3@10.129.17.122's password: ***********
```

OpenSSH can be configured and customized by editing the file `/etc/ssh/sshd_config` with a text editor. Here we can adjust settings such as the maximum number of concurrent connections, the use of passwords or keys for logins, host key checking, and more. However, it is important for us to note that changes to the OpenSSH configuration file must be done carefully.

For example, we can use SSH to securely log in to a remote system and execute commands or use tunneling and port forwarding to tunnel data over an encrypted connection to verify network settings and other system settings without the possibility of third parties intercepting the transmission of data and commands.

NFS

Network File System (`NFS`) is a network protocol that allows us to store and manage files on remote systems as if they were stored on the local system. It enables easy and efficient management of files across networks. For example, administrators use NFS to store and manage files centrally (for Linux and Windows systems) to enable easy collaboration and management of data. For Linux, there are several NFS servers, including NFS-UTILS (`Ubuntu`), NFS-Ganesha (`Solaris`), and OpenNFS (`Redhat Linux`).

It can also be used to share and manage resources efficiently, e.g., to replicate file systems between servers. It also offers features such as access controls, real-time file transfer, and support

for multiple users accessing data simultaneously. We can use this service just like FTP in case there is no FTP client installed on the target system, or NFS is running instead of FTP.

We can install NFS on Linux with the following command:

Install NFS

```shellsession
athane@htb[/htb]$ sudo apt install nfs-kernel-server -y
```

To check if the server is running, we can use the following command:

Server Status

```shellsession
athane@htb[/htb]$ systemctl status nfs-kernel-server

● nfs-server.service - NFS server and services
     Loaded: loaded (/lib/system/system/nfs-server.service; enabled; vendor preset:
enabled)
     Active: active (exited) since Sun 2023-02-12 21:35:17 GMT; 13s ago
    Process: 9234 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SUCCESS)
    Process: 9235 ExecStart=/usr/sbin/rpc.nfsd $RPCNFSDARGS (code=exited,
status=0/SUCCESS)
   Main PID: 9235 (code=exited, status=0/SUCCESS)
        CPU: 10ms
```

We can configure NFS via the configuration file `/etc/exports`. This file specifies which directories should be shared and the access rights for users and systems. It is also possible to configure settings such as the transfer speed and the use of encryption. NFS access rights determine which users and systems can access the shared directories and what actions they can perform. Here are some important access rights that can be configured in NFS:

Permissions	Description
`rw`	Gives users and systems read and write permissions to the shared directory.
`ro`	Gives users and systems read-only access to the shared directory.
`no_root_squash`	Prevents the root user on the client from being restricted to the rights of a normal user.
`root_squash`	Restricts the rights of the root user on the client to the rights of a normal user.
`sync`	Synchronizes the transfer of data to ensure that changes are only transferred after they have been saved on the file system.
`async`	Transfers data asynchronously, which makes the transfer faster, but may cause inconsistencies in the file system if changes have not been fully committed.

For example, we can create a new folder and share it temporarily in NFS. We would do this as follows:

Create NFS Share

```shellsession
cry0l1t3@htb:~$ mkdir nfs_sharing
cry0l1t3@htb:~$ echo '/home/cry0l1t3/nfs_sharing hostname(rw,sync,no_root_squash)' >> /etc/exports
cry0l1t3@htb:~$ cat /etc/exports | grep -v "#"

/home/cry0l1t3/nfs_sharing hostname(rw,sync,no_root_squash)
```

If we have created an NFS share and want to work with it on the target system, we have to mount it first. We can do this with the following command:

Mount NFS Share

```shellsession
cry0l1t3@htb:~$ mkdir ~/target_nfs
cry0l1t3@htb:~$ mount 10.129.12.17:/home/john/dev_scripts ~/target_nfs
cry0l1t3@htb:~$ tree ~/target_nfs

target_nfs/
├── css.css
├── html.html
├── javascript.js
├── php.php
└── xml.xml

0 directories, 5 files
```

So we have mounted the NFS share (`dev_scripts`) from our target (`10.129.12.17`) locally to our system in the mount point `target_nfs` over the network and can view the contents just as if we were on the target system. There are even some methods that can be used in specific cases to escalate our privileges on the remote system using NFS.

Web Server

Understanding the operation of web servers is essential for penetration testers, as these servers are integral to web applications and frequently serve as primary targets during security assessments. A web server is software that delivers data, documents, applications, and various functions over the Internet. It utilizes the Hypertext Transfer Protocol (`HTTP`) to transmit data to clients such as web browsers and to receive requests from these clients. The received data is then rendered as Hypertext Markup Language (`HTML`) within the client's browser, facilitating the creation of dynamic web pages that respond interactively to user requests. Consequently, a thorough comprehension of web server functionalities is vital for developing secure and efficient web applications and for maintaining overall system security. Among the most widely used web servers on Linux platforms are Apache, Nginx, Lighttpd, and Caddy, with Apache being particularly popular due to its broad compatibility with operating systems including Ubuntu, Solaris, and Red Hat Linux.

For penetration testers, web servers offer various utilities. They can be employed to facilitate file transfers, enabling testers to log in and interact with target systems through HTTP or HTTPS ports. Additionally, web servers can be leveraged to conduct phishing attacks by hosting replicas of target pages, thereby attempting to capture user credentials. Beyond these applications, web servers provide numerous other opportunities for testing and exploiting vulnerabilities within a network.

Apache web server has a variety of features that allow us to host a secure and efficient web application. Moreover, we can also configure logging to get information about the traffic on our server, which helps us analyze attacks. We can install Apache using the following command:

Install Apache Web Server

```shellsession
athane@htb[/htb]$ sudo apt install apache2 -y
```

For Apache2, to specify which folders can be accessed, we can edit the file `/etc/apache2/apache2.conf` with a text editor. This file contains the global settings. We can change the settings to specify which directories can be accessed and what actions can be performed on those directories.

Apache Configuration

```txt
<Directory /var/www/html>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
</directory>
```

This section specifies that the default `/var/www/html` folder is accessible, that users can use the `Indexes` and `FollowSymLinks` options, that changes to files in this directory can be overridden with `AllowOverride All`, and that `Require all granted` grants all users access to this directory. For example, if we want to transfer files to one of our target systems using a web

server, we can put the appropriate files in the `/var/www/html` folder and use `wget` or `curl` or other applications to download these files on the target system.

It is also possible to customize individual settings at the directory level by using the `.htaccess` file, which we can create in the directory in question. This file allows us to configure certain directory-level settings, such as access controls, without having to customize the Apache configuration file. We can also add modules to get features like `mod_rewrite`, `mod_security`, and `mod_ssl` that help us improve the security of our web application.

Python Web Server is a simple, fast alternative to Apache and can be used to host a single folder with a single command to transfer files to another system. To install Python Web Server, we need to install Python3 on our system and then run the following command:

Install Python & Web Server

```shellsession
athane@htb[/htb]$ sudo apt install python3 -y
athane@htb[/htb]$ python3 -m http.server
```

When we run this command, our Python Web Server will be started on the `TCP/8000` port, and we can access the folder we are currently in. We can also host another folder with the following command:

```shellsession
athane@htb[/htb]$ python3 -m http.server --directory /home/cry0l1t3/target_files
```

This will start a Python web server on the `TCP/8000` port, and we can access the `/home/cry0l1t3/target_files` folder from the browser, for example. When we access our Python web server, we can transfer files to the other system by typing the link in our browser and downloading the files. We can also host our Python web server on a port other than the default port:

```shellsession
athane@htb[/htb]$ python3 -m http.server 443
```

This will host our Python web server on port 443 instead of the default `TCP/8000` port. We can access this web server by typing the link in our browser.

VPN

A Virtual Private Network (`VPN`) functions like a secure, invisible tunnel that connects us to another network, allowing seamless and protected access as if we were physically present within it. This is achieved by establishing an encrypted tunnel between the client and the server, ensuring that all data transmitted through this connection remains confidential and safeguarded from unauthorized access.

Organizations primarily utilize VPNs to grant their employees secure access to the internal network without requiring them to be on-site. This flexibility enables employees to reach internal resources and applications from any location, enhancing productivity and mobility. Additionally, VPNs serve to anonymize internet traffic and block external intrusions, further bolstering security.

Among the most widely used VPN solutions for Linux servers are OpenVPN, L2TP/IPsec, PPTP, SSTP, and SoftEther. OpenVPN stands out as a popular open-source option compatible with various operating systems, including Ubuntu, Solaris, and Red Hat Linux. Administrators leverage OpenVPN to facilitate secure remote access to corporate networks, encrypt network traffic, and maintain user anonymity online.

For penetration testers, OpenVPN offers invaluable capabilities. It allows testers to securely connect to internal networks, especially when direct access is not feasible due to geographical constraints. By utilizing OpenVPN, penetration testers can perform comprehensive security assessments of internal systems, identifying and addressing potential vulnerabilities. The versatility of OpenVPN, with features such as encryption, tunneling, traffic shaping, network routing, and adaptability to dynamic network environments, makes it an essential tool in the arsenal of both network administrators and security professionals. We can install the server and client with the following command:

Install OpenVPN

```shellsession
athane@htb[/htb]$ sudo apt install openvpn -y
```

OpenVPN can be customized and configured by editing the configuration file `/etc/openvpn/server.conf`. This file contains the settings for the OpenVPN server. We can change the settings to configure certain features such as encryption, tunneling, traffic shaping, etc.

If we want to connect to an OpenVPN server, we can use the `.ovpn` file we received from the server and save it on our system. We can do this with the following command on the command line:

Connect to VPN

```shellsession
athane@htb[/htb]$ sudo openvpn --config internal.ovpn
```

After the connection is established, we can communicate with the internal hosts on the internal network.

Connect to HTB